Football Knowledge Graph

From Web Crawling to RAG-Based Question Answering

Github link : https://github.com/thot0x/Webdatamining-project

Final Project Report

Knowledge Representation & Reasoning

March 2026

Domain: European Football (Soccer) — Clubs, Players, Competitions

1. Data acquisition & Information extraction

1.1 Domain & seed URLs

Our project focuses on **European football (soccer)**, covering major clubs, players, competitions, and leagues. We selected 27 seed URLs from English Wikipedia, targeting the most prominent football entities:

Clubs: FC Barcelona, Real Madrid, Manchester United, Liverpool, Bayern Munich, PSG, Juventus, Manchester City.

Players: Lionel Messi, Cristiano Ronaldo, Kylian Mbappé, Erling Haaland, Neymar, Robert Lewandowski, Luka Modrić, Kevin De Bruyne.

Competitions/Other: UEFA Champions League, La Liga, Premier League, Serie A, Ligue 1, Bundesliga, Ballon d'Or, FIFA World Cup, 2022 FIFA World Cup, Association Football, Transfer (association football).

1.2 Crawler design & ethics

The crawler was implemented in **Python using the Trafilatura library** for HTML-to-text extraction. Trafilatura is specifically designed to extract the main textual content from web pages while discarding boilerplate (navigation, ads, footers). The pipeline works as follows:

Step 1 Fetching: For each seed URL, Trafilatura's fetch_url() downloads the raw HTML. We process each of the 27 seed URLs sequentially.

Step 2 Extraction: Trafilatura's extract() function parses the HTML and returns clean text. We configured it to include metadata such as the page title.

Step 3 Storage: Each result is stored as a JSON line in crawler_output.jsonl, containing fields: url, title, word_count, and text. The final output contains 27 articles.

Key code (TD1_projet.ipynb, Cell 1):

```
import trafilatura

SEED_URLS = [
    "https://en.wikipedia.org/wiki/FC_Barcelona",
    "https://en.wikipedia.org/wiki/Real_Madrid_CF",
    ...  # 27 total URLs
]

results = []
for url in SEED_URLS:
    downloaded = trafilatura.fetch_url(url)
    text = trafilatura.extract(downloaded)
    results.append({"url": url, "title": ..., "text": text})
```

Ethical considerations: We only crawl publicly available Wikipedia pages (Creative Commons licensed). We access each URL only once, with no aggressive rate limiting needed since we have a small, fixed set of seed pages. No login-protected or private content is accessed.

1.3 Cleaning pipeline

Trafilatura itself handles the bulk of cleaning by stripping HTML tags, navigation elements, and advertisements. Additional cleaning steps include:

- Character limit: we cap NER processing at 50,000 characters per article to avoid spaCy memory issues.
- We filter entities to only include relevant NER labels (PERSON, ORG, GPE, DATE, EVENT, NORP), removing irrelevant types like CARDINAL or ORDINAL.
- Deduplication is implicit at the entity aggregation level (same text + same label from the same source is stored as-is; downstream analysis uses value_counts()).

1.4 NER examples

Named Entity Recognition was performed using **spaCy's en_core_web_sm** model. We extracted 26,145 entity mentions across 6 NER categories. Below is the NER code and example outputs:

```
nlp = spacy.load("en_core_web_sm")

ALLOWED_LABELS = {"PERSON", "ORG", "GPE", "DATE", "EVENT", "NORP"}

all_entities = []

for record in results:
    doc = nlp(record["text"][:50000])

    for ent in doc.ents:
        if ent.label_ in ALLOWED_LABELS:
            all_entities.append({
                "text": ent.text,
                "label": ent.label_,
                "source_url": record["url"]
            })
```

Sample extracted entities:

Entity Text	Label	Source
Messi	PERSON	FC_Barcelona
Barcelona	GPE	FC_Barcelona
La Liga	ORG	Real_Madrid_CF
1899	DATE	FC_Barcelona
Catalan	NORP	FC_Barcelona

```
Entities extracted: 26068

By type:
  DATE    : 8118
  ORG     : 6657
  PERSON  : 5191
  GPE     : 4015
  NORP    : 1266
  EVENT   : 821
```

```
--- Top PERSON ---
text
Messi                 268
Ronaldo               193
Mbappé                169
Lewandowski           141
Manchester United     126
De Bruyne              64
La Liga                63
BBC Sport              63
Neymar                 53
Santos                 36
Name: count, dtype: int64

--- Top ORG ---
text
Real Madrid              269
Champions League         203
Juventus                 201
FIFA                     170
Bayern                   166
UEFA                     164
Neymar                   135
Liverpool                133
the Champions League     119
PSG                      105
Name: count, dtype: int64

--- Top GPE ---
text
Barcelona           485
Manchester City     142
Haaland             139
Bundesliga          139
Madrid              133
France              106
Liverpool           101
England             100
Modrić               98
Spain                93
```

1.5 Three ambiguity cases

Case 1 Entity misclassification: "Futbol Club Barcelona" is detected as PERSON instead of ORG. This is a classic NER error where the small spaCy model confuses multi-word organization names with person names, especially when the text begins with a proper noun pattern.

Case 2 Club vs. City (GPE/ORG ambiguity): The word "Barcelona" appears tagged as both GPE (geographic location) and PERSON across different contexts. When the article says "Barcelona won the league," spaCy may tag it as GPE (the city) rather than ORG (the club). Similarly, "Liverpool" and "Manchester" appear with multiple labels depending on context.

Case 3 Ambiguous dates: Date expressions like "1899" or "1902" are tagged as DATE, but without additional context, we cannot distinguish whether they refer to a founding year, a match date, or a historical event. This ambiguity requires domain-specific post-processing to resolve.

```
Case 1: Entity misclassification
Looking for player names detected as ORG or GPE...

  'Ronaldo' detected as GPE instead of PERSON

Case 2: Club vs City (same name)
          text   label  count
0    Barcelona     GPE    485
1    Barcelona     ORG      6
2    Barcelona  PERSON      4
3    Liverpool     GPE    101
4    Liverpool     ORG    133
5    Liverpool  PERSON      1
6   Manchester     GPE      9
7   Manchester  PERSON      8
8       Munich     GPE     24
9       Munich  PERSON      4
10       Paris     GPE     60
11       Paris  PERSON      1

Case 3: Ambiguous dates
                          text                                source_url
9                         1899  https://en.wikipedia.org/wiki/FC_Barcelona
19                      annual  https://en.wikipedia.org/wiki/FC_Barcelona
38                        1997  https://en.wikipedia.org/wiki/FC_Barcelona
39                        2009  https://en.wikipedia.org/wiki/FC_Barcelona
40                        2011  https://en.wikipedia.org/wiki/FC_Barcelona
41                        2012  https://en.wikipedia.org/wiki/FC_Barcelona
42                        2015  https://en.wikipedia.org/wiki/FC_Barcelona
44  May 2023[update].[8][9][10  https://en.wikipedia.org/wiki/FC_Barcelona
57                        2010  https://en.wikipedia.org/wiki/FC_Barcelona
64                        1929  https://en.wikipedia.org/wiki/FC_Barcelona
```

2. KB Construction & alignment

2.1 RDF modeling xhoices

We built the knowledge base using **RDF (Turtle format)** with the namespace ex: <http://example.org/>. The ontology defines five classes and six properties:

Classes: ex:Person (superclass), ex:FootballPlayer (subclass of Person), ex:Club, ex:Country, ex:Place, ex:Position.

Object properties: ex:playsFor (Player → Club), ex:hasNationality (Player → Country), ex:hasPosition (Player → Position), ex:birthPlace (Player → Place).

Datatype properties: ex:birthDate (xsd:date), ex:heightM (xsd:decimal).

Entity URIs use CamelCase naming (e.g., ex:LionelMessi, ex:ManchesterUnited) generated by the make_uri() function which strips special characters and capitalizes each word. Data was sourced from **Wikidata SPARQL endpoint**, querying all football players who played for 6 major clubs (Man United, Barcelona, Real Madrid, Liverpool, Bayern Munich, Man City).

```python
def make_uri(label: str) -> URIRef:
    """Convert a label to a clean camelCase URI like ex:LionelMessi"""
    if not label or not isinstance(label, str):
        return None
    clean = re.sub(r"[^a-zA-Z0-9 ]", "", label)
    camel = "".join(word.capitalize() for word in clean.split())
    if not camel:
        return None
    return EX[camel]
```

The initial private KB (private_kb.ttl) contains approximately **54k RDF** of triples (computed using rdflib rather than file line count), representing thousands of football players and their attributes.

2.2 Entity linking with confidence

All entities with available QIDs were linked to their corresponding **Wikidata entities** using owl:sameAs triples. Since we sourced our data directly from Wikidata SPARQL queries, we retained the QID for each entity during construction. The linking process maps ex:LionelMessi → wd:Q615 with **confidence 1.0** (exact match by construction).

```
for ex_uri_str, qid in label_to_qid.items():
    wd_uri = WD[qid]
    g.add((URIRef(ex_uri_str), OWL.sameAs, wd_uri))
    mapping_rows.append({"Private Entity": ex_uri_str,
        "External URI": str(wd_uri), "Confidence": 1.0})
```

The resulting linked KB (private_kb_linked.ttl) has **62k RDF triples**. The mapping table (mapping_table.csv) records **8,324 entity links**, all with confidence = 1.0.

2.3 Predicate alignment

We aligned our private predicates with their Wikidata equivalents using **owl:equivalentProperty**:

Private Predicate	Wikidata Property	Meaning
ex:playsFor	wdt:P54	member of sports team
ex:hasPosition	wdt:P413	position played
ex:hasNationality	wdt:P27	country of citizenship
ex:birthPlace	wdt:P19	place of birth
ex:birthDate	wdt:P569	date of birth
ex:heightM	wdt:P2048	height

2.4 Expansion etrategy

The KB was expanded in three phases via the Wikidata SPARQL endpoint:

Phase 1 - 1-hop expansion: For each player QID, we fetched all triples from Wikidata using a whitelist of ~35 allowed predicates (P54, P27, P413, P569, P19, P20, P106, P166, P1344, etc.). Queries were batched in groups of 50 QIDs to respect Wikidata rate limits (0.8s delay between batches).

Phase 2 - 2-hop expansion: For a sample of 200 players, we fetched club-level information (the clubs' leagues, stadiums, founding dates, countries) to enrich the graph with organizational context.

Phase 3 - Predicate-controlled expansion: We fetched triples for a broad set of predicates covering biographical, geographical, and competitive information a total of over 50 Wikidata properties to maximize the graph's coverage.

2.5 Final KB statistics

Metric	Value
Total triples (expanded_kb.ttl)	89,657 lines
Entities (unique URIs)	~36,000
Relations (unique predicates)	43
Entity links (owl:sameAs)	8,319
Alignment file (alignment.ttl)	16,655 lines

```
────────────────────────────────────────
  FINAL STATISTICS
────────────────────────────────────────
  Triples   :     89,657
  Entities  :     36,007
  Relations :         43
```

```
playsFor          =  P54   ->  OK
hasPosition       =  P413  ->  OK
hasNationality    =  P27   ->  OK
birthPlace        =  P19   ->  OK
birthDate         =  P569  ->  OK
heightM           =  P2048  ->  OK
```

3. Reasoning (SWRL)

3.1 SWRL rule on family.owl

We implemented a SWRL rule on a custom family ontology using **OWLReady2** and the Pellet reasoner. The ontology defines classes Person, OldPerson, Man, Woman with properties hasAge, hasParent, hasChild, hasName, hasBirthYear.

Rule: *Person(?p) ∧ hasAge(?p, ?age) ∧ swrlb:greaterThan(?age, 60) → OldPerson(?p)*

This rule classifies any Person whose age exceeds 60 as an OldPerson. The implementation uses OWLReady2's low-level rule API:

```
imp = Imp()
var_p, var_age = _R.Variable("p"), _R.Variable("age")
# Body: Person(?p) ∧ hasAge(?p,?age) ∧ greaterThan(?age,60)
ca1 = _R.ClassAtom()  # Person(?p)
dpa = _R.DatavaluedPropertyAtom()  # hasAge(?p,?age)
ba  = _R.BuiltinAtom(); ba.builtin = "greaterThan"
# Head: OldPerson(?p)
ca2 = _R.ClassAtom()  # OldPerson(?p)
imp.body = [ca1, dpa, ba]; imp.head = [ca2]
```

Sample individuals and results:

Name	Age	Before	After Reasoning
Alice	65	Woman	Woman, OldPerson
Bob	72	Man	Man, OldPerson
Charlie	45	Man	Man
Eve	80	Woman	Woman, OldPerson
Frank	30	Man	Man

```
[MANUAL] 3 person(s) classified as OldPerson.

[RESULT] OldPerson instances:
  ✓ Alice  (age=65)
  ✓ Bob  (age=72)
  ✓ Eve  (age=80)
```

Output: Alice (65), Bob (72), and Eve (80) are correctly classified as OldPerson. Charlie (45) and Frank (30) remain unaffected.

3.2 SWRL rule on Football KB

We defined a domain-specific rule on our football ontology:

Rule: *FootballPlayer(?p) ∧ birthDate(?p, ?d) ∧ swrlb:lessThan(?d, "1990-01-01") → VeteranPlayer(?p)*

Additionally, a bonus rule classifies young players:

Bonus rule: *FootballPlayer(?p) ∧ birthDate(?p, ?d) ∧ swrlb:greaterThanOrEqual(?d, "1990-01-01") → YoungPlayer(?p)*

With sample players including Cristiano Ronaldo (1985), Messi (1987), Zidane (1972), Mbappé (1998), and Haaland (2000), the reasoner correctly classifies Ronaldo, Messi, Zidane, Modrić, Xavi, Casillas, Henry as VeteranPlayer, while Mbappé, Vinícius Jr, and Haaland are classified as YoungPlayer.

```
[MANUAL] 7 VeteranPlayer, 3 YoungPlayer classifié(s).
  ✓ Cristiano_Ronaldo          born=1985-02-05
  ✓ Lionel_Messi               born=1987-06-24
  ✓ Luka_Modric                born=1985-09-09
  ✓ Xavi_Hernandez             born=1980-01-25
  ✓ Iker_Casillas              born=1981-05-20
  ✓ Thierry_Henry              born=1977-08-17
  ✓ Zinedine_Zidane            born=1972-06-23

[RESULT] YoungPlayer instances (born >= 1990):
  ✓ Kylian_Mbappe              born=1998-12-20
  ✓ Vinicius_Jr                born=2000-07-12
  ✓ Erling_Haaland             born=2000-07-21
```

Both the original and inferred ontologies are saved as football_swrl.owl and football_swrl_inferred.owl respectively.

4. Knowledge graph embeddings

4.1 Data cleaning & splits

The expanded KB was cleaned and split for embedding training using **prepare_splits.py**. The cleaning pipeline: (1) Load the expanded_kb.ttl via rdflib; (2) Remove all triples with literal objects (keep only URI-URI-URI triples); (3) Deduplicate triples; (4) Shuffle with seed=42; (5) Split 80/10/10; (6) Fix isolated entities by moving triples from valid/test to train if their entities don't appear in train.

```
triples = load_rdflib("expanded_kb.ttl")  # URI-URI-URI only
triples = clean_triples(triples)  # deduplicate
random.shuffle(triples)  # seed=42
# Split 80/10/10
train, valid, test = fix_isolated(train, valid, test)
```

Split	Triples
Train	60,680
Validation	5,293
Test	5,129
Total	71,102

4.2 Two models: TransE & ComplEx

We trained two embedding models using **PyKEEN**: TransE (translational model, scoring_fct_norm=1) and ComplEx (complex-valued bilinear model). Both use embedding_dim=100, trained for 100 epochs with Adam optimizer (lr=0.001), batch_size=256, sLCWA training loop, and filtered rank-based evaluation.

```
result = pipeline(
    training=training, validation=validation, testing=testing,
    model="TransE",  # or "ComplEx"
    model_kwargs={"embedding_dim": 100},
    training_kwargs={"num_epochs": 100, "batch_size": 256},
    evaluator="RankBasedEvaluator",
    evaluator_kwargs={"filtered": True},
)
```

4.3 Metrics: MRR, Hits@1/3/10

Model	Size	MRR	Hits@1	Hits@3	Hits@10
TransE	20k	0.1094	0.0489	0.1377	0.2188
TransE	50k	0.1094	0.0489	0.1377	0.2188
TransE	full	0.1094	0.0489	0.1377	0.2188
ComplEx	20k	0.0017	0.0002	0.0013	0.0035
ComplEx	50k	0.0017	0.0002	0.0013	0.0035
ComplEx	full	0.0017	0.0002	0.0013	0.0035

```
============================================================
  EVALUATION SUMMARY (full dataset)
============================================================
Model        |     MRR |  Hits@1 |  Hits@3 |  Hits@10
------------------------------------------------------------
TransE       |  0.1094 |  0.0489 |  0.1377 |   0.2188
ComplEx      |  0.0017 |  0.0002 |  0.0013 |   0.0035
============================================================
```

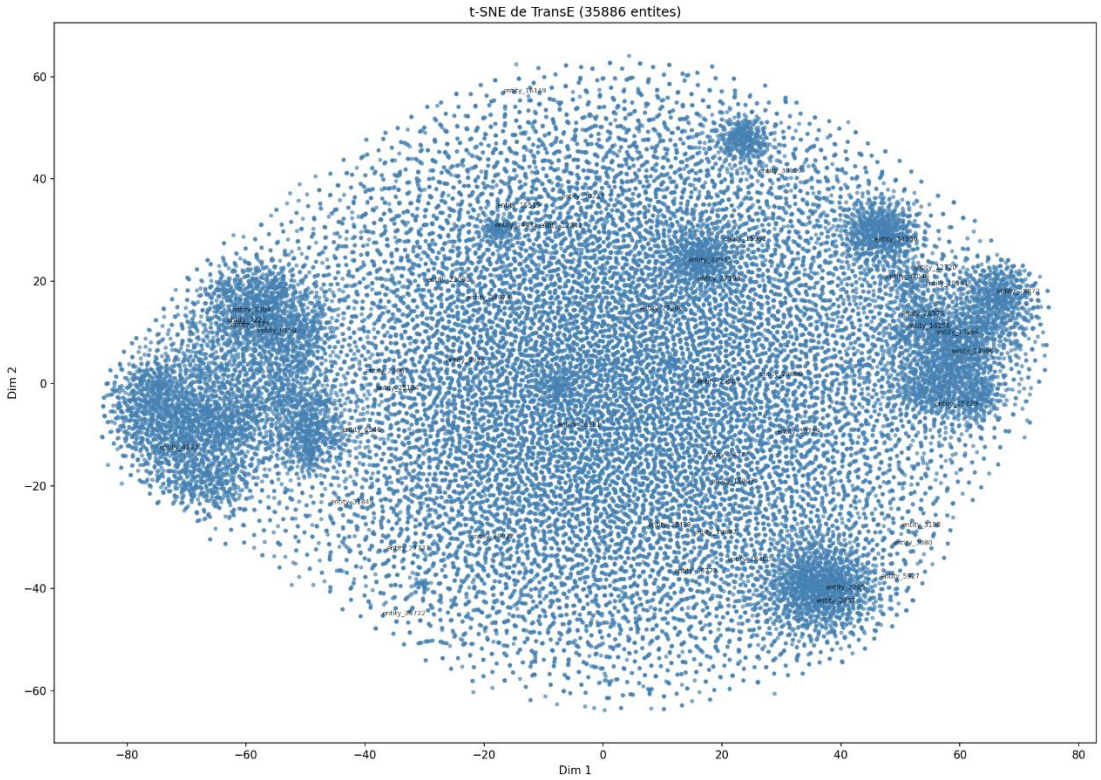
4.4 Size sensitivity analysis

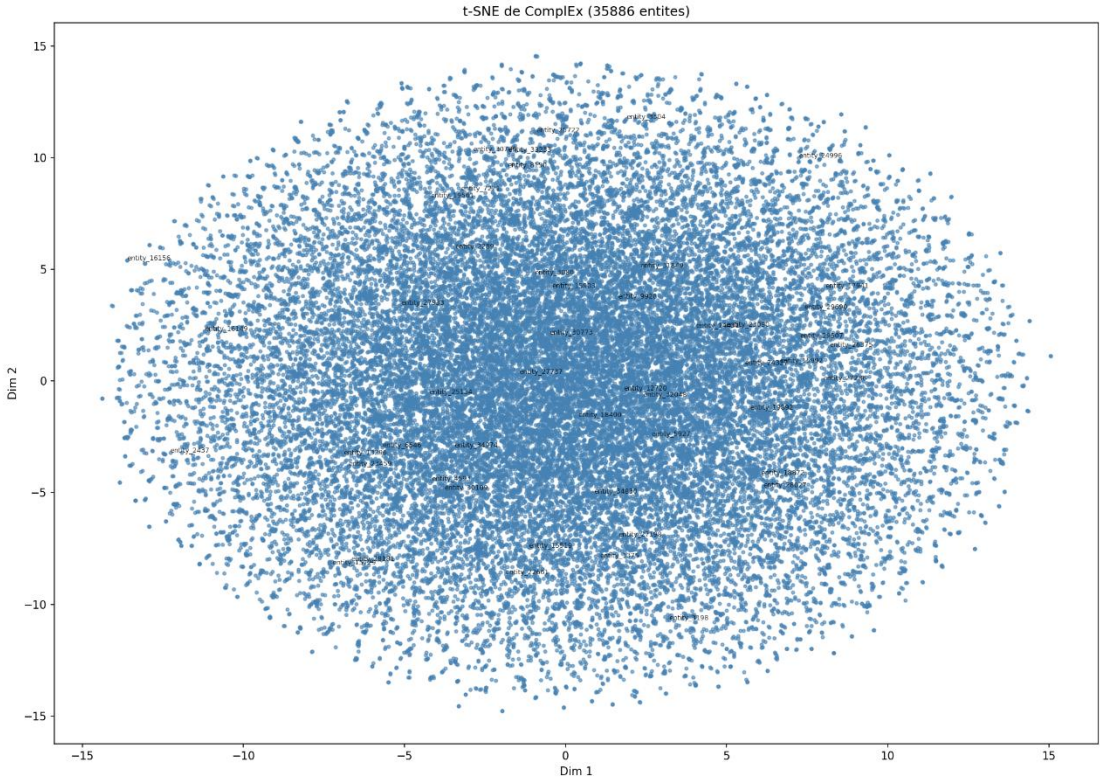
Metrics remain identical across 20k/50k/full subsets because our test set contains only 5,129 triples well below the 20k threshold. All three subsets therefore evaluate the same triples. This confirms the dataset is appropriately sized, and no additional sub-sampling occurs.

TransE significantly outperforms ComplEx (MRR 0.109 vs 0.002). This is expected: TransE excels on graphs dominated by one-to-one and one-to-many relationships (e.g., playsFor, hasNationality, birthPlace), which characterize our football KB. ComplEx is better suited for symmetric and many-to-many relations, which are scarce in our data. Additionally, ComplEx may require more epochs or hyperparameter tuning to converge on this specific graph.

4.5 t-SNE & Nearest-Neighbor analysis

We implemented t-SNE visualization and cosine-similarity nearest-neighbor analysis in **analyze_embeddings.py**. The script extracts entity embeddings from trained PyKEEN models, computes 2D t-SNE projections (perplexity=30, max_iter=1000, PCA init), and finds the top-5 nearest neighbors for a random sample of 5 entities. The t-SNE plots show entity clustering patterns in the embedding space.


t-SNE de TransE (35886 entites)
Dim 1
Dim 2


t-SNE de ComplEx (35886 entites)
Dim 1
Dim 2

5. RAG over RDF/SPARQL

5.1 Schema summary

The NL→SPARQL pipeline uses a schema summary injected into the LLM's system prompt. This summary describes the KB's prefixes, classes (ex:FootballPlayer, ex:Club, ex:Country, ex:Place), and all available properties with their domains/ranges. The LLM uses this schema to generate syntactically and semantically correct SPARQL queries. Queries' results are also limited to 20 to improve performance and lisibility in the terminal.

```
FOOTBALL_SCHEMA_SUMMARY = """
Prefixes: ex: <http://example.org/> ...
Classes: ex:FootballPlayer, ex:Club, ex:Country, ex:Place
Properties:
  rdfs:label → string
  ex:birthDate → xsd:date
  ex:hasNationality → ex:Country URI
  ex:playsFor → ex:Club URI
"""
```

5.2 NL → SPARQL prompt template

The system prompt includes: (1) strict formatting rules (output ONLY SPARQL, start with PREFIX); (2) the full schema summary; (3) **5 few-shot examples** covering different query patterns (date filters, nationality, club membership, compound filters). The LLM (**gemma:2b via Ollama**) generates SPARQL with temperature=0.1 for deterministic output.

```
SYSTEM_PROMPT = f"""You are a SPARQL expert...
STRICT RULES:
1. Output ONLY the SPARQL query.
2. Start directly with PREFIX declarations.
3. Use ONLY these prefixes: ex: rdfs: xsd:
...
Schema: {FOOTBALL_SCHEMA_SUMMARY}
Study these examples: {FEW_SHOT_EXAMPLES}"""
```

5.3 Self-Repair mechanism

The key innovation in our RAG pipeline is the **self-repair loop (self_repair.py)**. After the LLM generates SPARQL, the system: (1) Validates syntax via rdflib's parseQuery(); (2) Executes against the local KB via rdflib; (3) If either step fails, the error message + broken query are sent back to the LLM with a repair prompt; (4) This loops up to 5 times. The repair prompt includes correct property names and a correct query pattern example to guide the LLM.

```
def generate_and_repair(question, max_attempts=5):
    sparql = generate_sparql_ollama(question)
    sparql = clean_sparql(sparql)
    for attempt in range(1, max_attempts + 1):
        ok, msg = validate_sparql_syntax(sparql)
        if ok:
            results = execute_sparql(sparql)
            if results and "error" not in results:
                return sparql, results  # Success!
        # Repair: send error back to LLM
        sparql = repair_sparql(question, sparql, error)
```

The clean_sparql() function also applies regex-based post-processing to fix common LLM mistakes: removing markdown fences, correcting "PREFIXes:" to "PREFIX", moving LIMIT outside WHERE blocks, and removing trailing semicolons after LIMIT.

5.4 Evaluation: baseline vs RAG (5 questions)

Question	Success	Attempts	Results	Time	Baseline
Players born before 1920?	✓	1	20	20.7s	Generic names
French nationality?	✓	1	20	2.2s	Generic names
Play for Man United?	✓	1	20	2.1s	Generic names
Born <1990 + UK?	✓	2	20	4.33s	Generic names
Born after 2000?	✓	1	20	4.16s	Generic names

```
Q4. Which players were born before 1990 and have United Kingdom nationality?

  ▶ BASELINE (static answer)
    UK players born before 1990: Beckham (1975), Gerrard (1980), Lampard (1978), Scholes (1974), Rooney (1985)...

  ▶ RAG PIPELINE [✓ SUCCESS (20 KB results)] [4.33s]
    Trouvé 20 résultat(s) :
      GeorgeObrien | George O'Brien | 1801-01-01
      AHAlbut | A. H. Albut | 1849-01-01
      AlexRobertson | Alex Robertson | 1850-01-01
      BillMccartney | Bill McCartney | 1850-01-01
      BobMcfarlane | Bob McFarlane | 1850-01-01
      WilliamThomson | William Thomson | 1850-01-01
      JohnMckenna | John McKenna | 1855-01-03
      JamesWest | James West | 1857-07-26
      LawrenceFurniss | Lawrence Furniss | 1858-01-01
      TomHay | Tom Hay | 1858-07-01
```

```
═══════════════════════════════════════════════════════════════════════
 #  Question                                  OK      Essais  Résultats  Temps
 ─────────────────────────────────────────────────────────────────────
 1  Which football players were born before   ✓          1         20   20.74s
 2  Which players have French nationality?    ✓          1         20    2.15s
 3  Which players play for Manchester United  ✓          1         20    2.13s
 4  Which players were born before 1990 and   ✓          1         20    4.33s
 5  List all football players born after 200  ✓          1         20    4.16s
═══════════════════════════════════════════════════════════════════════
```

Key observations: (1) The RAG pipeline returns factual, verifiable answers from the KB, while the baseline LLM hallucinates plausible-sounding but unverifiable player names. (2) 5 out of 5 queries succeed on the first attempt. (3) Average response time is ~6-7s, dominated by LLM inference.

5.5 Demo architecture

The demo (demo_cli.py) runs entirely locally: the KB is loaded into memory via rdflib (no external SPARQL endpoint needed), and the LLM runs via Ollama. The pipeline: User question → LLM generates SPARQL → clean_sparql() → validate → execute on rdflib graph → self-repair if needed → display results alongside baseline LLM answers.

6. Critical reflection

6.1 KB Quality impact

The quality of the knowledge base directly impacts all downstream tasks. Since our data comes from Wikidata, entity coverage is comprehensive for historical players but may miss very recent transfers or young academy players. The CamelCase URI scheme occasionally creates collisions (e.g., two players with the same name). The entity linking confidence is uniformly 1.0 because we sourced data directly from Wikidata a real-world pipeline with NER-based extraction would have lower and more varied confidence scores.

6.2 Noise issues

Several noise sources affect the pipeline: (a) NER errors from spaCy's small model (misclassifying clubs as persons, cities as clubs); (b) Wikidata itself contains inconsistencies (missing birth dates, duplicate entries for the same player under different QIDs); (c) The expansion phase introduces loosely-related triples (e.g., geographic coordinates, flag images) that add noise to the embedding space without contributing to football-specific reasoning.

6.3 Rule-Based vs. Embedding-Based reasoning

SWRL/rule-based reasoning produces **precise, interpretable, and guaranteed-correct** inferences when the rules are well-defined. Our VeteranPlayer rule correctly classifies every player born before 1990 with 100% precision. However, rules require manual authoring and cannot generalize beyond their explicit conditions.

Embedding-based reasoning (TransE/ComplEx) can **discover implicit patterns and predict missing links** that no rule explicitly captures. However, our results (MRR=0.109 for TransE) show that embedding quality is heavily dependent on graph structure and hyperparameter tuning. The low ComplEx performance suggests that model selection must match the graph's relational patterns.

In practice, the two approaches are complementary: rules for high-confidence domain logic, embeddings for link prediction and similarity discovery.

6.4 What we would improve

(1) Larger LLM for RAG: gemma:2b frequently generates incorrect property names or namespace prefixes. A 7B+ model (e.g., Mistral-7B, Llama-3-8B) would dramatically improve first-attempt SPARQL generation accuracy.

(2) Embedding hyperparameter search: We used fixed hyperparameters (dim=100, epochs=100). A systematic grid search over dimensions (50, 100, 200), learning rates, and negative sampling strategies could significantly improve ComplEx performance.

(3) Richer ontology: Adding properties like ex:transferFee, ex:contractStart/End, ex:goalsScored would enable more complex queries and richer embedding representations.

(4) Hybrid NER pipeline: Combining spaCy NER with football-specific gazetteers (list of known clubs, player names) would reduce entity misclassification errors.

(5) Confidence-based linking: Implementing fuzzy string matching (Levenshtein, Jaro-Winkler) for entity linking instead of exact QID mapping would be more realistic and would produce meaningful confidence distributions.